

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2413 — Bases de Datos — Sección 3 — 2'2026

TAREA 3

Tarea:	Optimización de consultas.
Publicación:	Martes 27 de Octubre.
Entrega:	**Viernes 6 de Noviembre a las 23:59 horas**.

Modalidad de trabajo

Esta tarea es de carácter **individual**. El desarrollo, la implementación y la redacción del informe deben ser realizados exclusivamente por cada estudiante. Cualquier intento de plagio o copia será sancionado de acuerdo con las normativas académicas de la Escuela de Ingeniería.

Objetivos de la tarea

1. Leer e interpretar planes de ejecución en PostgreSQL para identificar qué ralentiza una consulta.
2. Decidir qué índices y vista materializada crear bajo recursos limitados.
3. Reescribir consultas SQL ineficientes.

Contexto: e-commerce de librería

En esta tarea se trabajará con una base de datos de una librería. Estos datos son generados, pero imitan el comportamiento de una tienda real: algunos clientes concentran muchos pedidos, unos libros se venden considerablemente más que el resto y las ventas aumentan en noviembre y diciembre. La base contiene cerca de 2,4 millones de pedidos entre los años 2022 y 2025.

Los datos representan el ciclo de venta de la librería, con las siguientes entidades lógicas representadas por relaciones (esto es, tablas):

- **Transacciones:** `orders` es el núcleo del modelo, donde cada fila es un pedido con su fecha, su estado y su monto total. `order_items` detalla los libros que forman parte de cada pedido.
- **Clientes:** `users` contiene a los usuarios de la librería.
- **Catálogo:** `books` contiene los libros a la venta, `authors` a sus autores e `inventory` el stock de cada libro en cada bodega.
- **Experiencia:** `reviews` contiene las reseñas que los usuarios escriben sobre los libros.

Archivos de la tarea

Para el desarrollo de esta tarea, contará con un archivo `.zip` el cual incluye los archivos para crear y poblar la base de datos, el workload de consultas y scripts de apoyo:

- **schema.sql**, **load.sql:** Crea las tablas y carga los datos, respectivamente.
- **workload.sql:** Conjunto de quince consultas a optimizar.
- **run_workload.py, explain_plans.py, apply_indexes.py, apply_matview.py, check_results.py**, **compare_times.py:** Scripts de apoyo para el desarrollo de la tarea (detallados en `README.md`).

Adicionalmente, el `.zip` incluye siete archivos `.csv` dentro de una carpeta `data/`. La descripción de los archivos `.csv` es la siguiente:

1. **users.csv:** Es el archivo que contiene toda la información sobre los usuarios de la librería (`users`), específicamente:
   - `user_id`: ID único del usuario.
   - `email`: Correo del usuario, el cual es único.
   - `signup_date`: Fecha en que el usuario se registró.
   - `country`: Código del país del usuario (ej: CL), el cual puede ser nulo.
   - `is_premium`: Indica si el usuario tiene una suscripción premium o no.
2. **authors.csv:** Es el archivo que contiene los autores (`authors`), específicamente:
   - `author_id`: ID único del autor.
   - `name`: Nombre del autor.
   - `country`: Código del país del autor, el cual puede ser nulo.
   - `birth_year`: Año de nacimiento del autor.
3. **books.csv:** Es el archivo que contiene los libros que tiene la librería (`books`), específicamente:
   - `book_id`: ID único del libro.
   - `title`: Título del libro.
   - `author_id`: ID del autor del libro.
   - `genre`: Género del libro (hay quince géneros distintos).
   - `price`: Precio actual del libro.
   - `publication_year`: Año de publicación.
   - `page_count`: Cantidad de páginas, la cual puede ser nula.
   - `isbn`: Código ISBN del libro, el cual no sigue un estándar para el formato y puede contener guiones o no.
4. **inventory.csv:** Es el archivo que contiene el stock de cada libro en cada bodega (`inventory`), específicamente:
   - `book_id`: ID del libro.
   - `warehouse_id`: ID de la bodega (hay ocho bodegas distintas).
   - `stock_count`: Unidades disponibles del libro en esa bodega.

- `last_updated`: Fecha y hora de la última actualización del stock.

5. **orders.csv:** Es el archivo que contiene la información sobre los pedidos (`orders`), específicamente:
   - `order_id`: ID único del pedido.
   - `user_id`: ID del usuario que hizo el pedido.
   - `order_date`: Fecha y hora del pedido.
   - `status`: Estado del pedido, el cual puede ser `pending`, `shipped`, `delivered`, `cancelled` o `refunded`.
   - `total_amount`: Monto total del pedido.
6. **order_items.csv:** Es el archivo que contiene los ítems de cada pedido (`order_items`), específicamente:
   - `order_id`: ID del pedido.
   - `book_id`: ID del libro.
   - `quantity`: Unidades compradas de ese libro.
   - `unit_price`: Precio unitario del libro.
7. **reviews.csv:** Es el archivo que contiene la información sobre las reseñas de los libros (`reviews`), específicamente:
   - `review_id`: ID único de la reseña.
   - `book_id`: ID del libro reseñado.
   - `user_id`: ID del usuario que escribió la reseña.
   - `rating`: Puntuación de una a cinco estrellas.
   - `review_text`: Texto de la reseña el cual puede ser nulo.
   - `review_date`: Fecha y hora de la reseña.
   - `helpful_count`: Cuántos usuarios marcaron la reseña como útil.

PostgreSQL y librerías

En esta tarea ocuparemos el motor de bases de datos **PostgreSQL 18**, junto a su CLI `psql`. Es obligatorio usar esta versión, ya que los tiempos y los planes de ejecución pueden cambiar entre versiones. Puede verificar su versión con el comando `psql --version`. Para los scripts de apoyo se necesita solamente la siguiente librería:

- `psycopg2`: Librería para conectarse a PostgreSQL directamente desde Python.

Para instalarla basta con ejecutar el comando: `pip install psycopg2-binary`.

Problemas a resolver

El objetivo de esta tarea es que el workload se ejecute **al menos 8 veces más rápido** que en su línea base, es decir, lograr un *speedup* de 8× o más, donde

$$\textit{speedup} = \frac{\text{tiempo total del workload base}}{\text{tiempo total del workload final}}.$$

Para lograrlo, la tarea está subdividida en **cinco partes** (cuatro evaluadas) que usted debe resolver, y en cada una se explica qué debe hacer.

Parte 0: Preparación de la base de datos y línea base

Esta primera parte se enfoca en dejar la base de datos preparada y medir cuánto tiempo se demora en ejecutar el workload como línea base (sin ninguna optimización). Cada mejora que se aplique se comparará contra este tiempo inicial. Para esto debe seguir los siguientes pasos:

1. **Crear y poblar la base de datos:** Desde la carpeta de la tarea, cree la base de datos en PostgreSQL. Para esto ejecute los archivos `schema.sql` (crea las tablas) y `load.sql` (carga los datos desde `data/`).

   ```
   createdb bdd_t3
   psql -d bdd_t3 -f schema.sql
   psql -d bdd_t3 -f load.sql
   ```

   Sea paciente dado que la carga puede tomar algunos minutos.

2. **Medir el tiempo del workload (línea base):** Ejecute el workload base con el script `run_workload.py`.

   ```
   python3 run_workload.py --db bdd_t3 --csv tiempos_base.csv
   ```

   El script ejecuta las quince consultas dos veces seguidas (ejecución en caliente) y reporta los tiempos de la segunda ejecución de cada consulta. La primera ejecución es necesaria para asegurar que PostgreSQL cargue en memoria los índices y páginas básicas y así el tiempo medido de la segunda ejecución sea el de una ejecución habitual de un usuario (esto es, en caliente). El tiempo total se imprime en consola y el de cada consulta se guarda en el archivo `tiempos_base.csv`.

3. **Guardar los planes de ejecución:** Con el script `explain_plans.py` se obtienen los planes de cada consulta del workload base.

   ```
   python3 explain_plans.py --db bdd_t3 --out plans_base
   ```

   El script ejecuta `EXPLAIN ANALYZE` sobre cada consulta y guarda los resultados en la carpeta `plans_base/`, un archivo por consulta como `01.txt`, `02.txt`, etc.

Parte 1: Clasificación de las consultas

Esta parte se enfoca en entender por qué cada consulta tarda lo que tarda, antes de optimizar. Para esto debe leer cada consulta de `workload.sql` junto con su plan de ejecución (en `plans_base/`) y su tiempo (en `tiempos_base.csv`), y clasificarla en **una** de estas 4 categorías:

1. **Está bien como está:** La consulta es rápida y su plan no contiene trabajo innecesario. No requiere mejoras.
2. **Le falta un índice:** La consulta está bien escrita, pero su plan requiere recorrer una tabla completa para encontrar pocas filas. Un índice conveniente le permitiría acceder directamente a esas filas.
3. **Es un reporte pesado:** La consulta resume una gran cantidad de datos mediante agregaciones (por ejemplo, ventas totales por mes o por género). Está bien escrita, pero su respuesta depende de **muchas** filas, por lo que ningún índice evita leerlas. Se puede precalcular el resultado en una vista materializada.
4. **Está mal optimizada:** El plan encontrado hace mucho más trabajo del necesario. Existe un plan equivalente, que devuelve exactamente el mismo resultado y es mucho más eficiente.

Toda consulta del workload pertenece a exactamente una de estas categorías.

En su informe debe incluir una tabla con la clasificación de las quince consultas, indicando su tiempo en la línea base y una evidencia que justifique la categoría asignada (esta debe venir del plan de ejecución, no solamente del tiempo de la consulta).

Parte 2: Índices

Esta parte se enfoca en acelerar las consultas que les falta un índice. Usted puede crear **a lo más 3 índices**, por lo que debe decidir cuáles aportan más al workload dado.

1. **Definir los índices:** Escriba sus índices en un archivo llamado `indexes.sql`, con a lo más 3 sentencias `CREATE INDEX`. Puede elegir el tipo de cada índice: por defecto, PostgreSQL crea un índice B+ tree, y con `USING hash` crea un índice Hash. Por ejemplo:

   ```
   CREATE INDEX ix_authors_country ON authors (country);
   CREATE INDEX ix_authors_country_hash ON authors USING hash (country);
   ```

   La primera sentencia crea un índice B+ tree y la segunda un índice Hash sobre la misma columna. En su informe debe justificar el tipo de índice que eligió para cada caso.

2. **Aplicar los índices:** Con el script `apply_indexes.py`:

   ```
   python3 apply_indexes.py --db bdd_t3
   ```

3. **Medir y comparar:** Vuelva a medir el workload, guarde los planes y compare con la línea base.

   ```
   python3 run_workload.py --db bdd_t3 --csv tiempos_indices.csv
   python3 explain_plans.py --db bdd_t3 --out plans_indices
   python3 compare_times.py tiempos_base.csv tiempos_indices.csv
   ```

   El script `compare_times.py` muestra el tiempo total y por cada consulta, comparándolos con respecto a la línea base.

En su informe debe incluir, para cada índice, qué consultas acelera y por qué, y el impacto que tuvo, tanto en tiempo como en el plan de ejecución.

Consideración: Es posible que el tiempo total del workload no baje mucho, pero las consultas beneficiadas por los índices sí reducen su tiempo de ejecución drásticamente.

Parte 3: Vista materializada

Sobre vistas materializadas

Algunas consultas no se aceleran con índices: el costo no está en el acceso a los datos, sino en el cómputo (joins muy grandes, agregaciones que necesitan ver todas las filas, etc.). Para esas conviene precalcular el resultado y dejarlo guardado en disco, de modo que las consultas siguientes lo lean en vez de recalcularlo.

En PostgreSQL esto se hace con una vista materializada:

```
CREATE MATERIALIZED VIEW <nombre-vista> AS
<consulta SQL que produce el resultado a precalcular>;
```

PostgreSQL ejecuta la consulta una vez y guarda el resultado en disco como si fuera una relación más. De hecho, usted puede hasta crear un índice sobre esta tabla. Cuando los datos subyacentes cambian (por ejemplo, hay un `INSERT` a los datos), se debe recalcular ocupando el siguiente comando:

```
REFRESH MATERIALIZED VIEW <nombre-vista>;
```

Entre actualizaciones, los datos de la vista quedan almacenados y por lo tanto hay que mantenerlos, o sea, calcularlos de nuevo. Para esta tarea no es necesario usarlo, pues no haremos modificaciones a los datos.

Es importante notar que, al crear la vista materializada, esto no hace que las consultas existentes la usen automáticamente: hay que reescribir la consulta original para que la consuma. Por ejemplo, supongamos que usted tiene la siguiente consulta SQL:

```
SELECT COUNT(*)
FROM ( SELECT author_id
       FROM books
       GROUP BY author_id
       HAVING COUNT(*) > 50 ) sub;
```

pero observa que el tiempo de ejecución de la consulta es alto. Si creamos la vista materializada:

```
CREATE MATERIALIZED VIEW author_book_counts AS
SELECT author_id, COUNT(*) AS n_books FROM books GROUP BY author_id;
```

este comando creará una tabla pequeña con la cuenta por autor, que es muy útil para la consulta inicial. Para usar esta vista materializada hay que reescribir nuestra consulta inicial como:

```
SELECT COUNT(*)
FROM author_book_counts
WHERE n_books > 50;
```

Es posible verificar que la nueva consulta devuelve el mismo número que la consulta inicial, solo que ahora el resultado es obtenido directamente desde la vista materializada, haciendo nuestra consulta más eficiente.

Tarea a resolver

Esta parte se enfoca en acelerar los reportes pesados (consultas que resumen una gran cantidad de datos mediante agregaciones, ver Parte 1) usando vistas materializadas. Para esto, usted debe crear **exactamente una vista materializada** y reescribir sus consultas de la siguiente manera:

1. **Definir la vista:** Escriba en un archivo llamado `matview.sql` exactamente una sentencia `CREATE MATERIALIZED VIEW` y, si quiere, una sentencia `CREATE INDEX` sobre esta.
2. **Reescribir las consultas:** Copie el workload en un nuevo archivo llamado `workload_after.sql`. En este archivo debe reescribir las consultas que puedan usar su vista **sin cambiar sus etiquetas** (`-- ===== 07 =====`, etc.). Este nuevo workload aún debe tener quince consultas.
3. **Aplicar la vista y verificar:** Con el script `apply_matview.py` puede aplicar su vista materializada y con el script `check_results.py` puede verificar que sus nuevas consultas retornen exactamente los mismos resultados.

   ```
   python3 apply_matview.py --db bdd_t3
   python3 check_results.py --db bdd_t3
   ```

   El script `apply_matview.py` borra la vista que haya creado antes, aplicando la nueva.
4. **Medir y comparar:** Mida el workload reescrito, guarde sus planes y compárelo con las mediciones anteriores.

```
python3 run_workload.py \
        --db bdd_t3 --workload workload_after.sql --csv tiempos_vista.csv
python3 explain_plans.py \
        --db bdd_t3 --workload workload_after.sql --out plans_vista
python3 compare_times.py tiempos_base.csv tiempos_indices.csv tiempos_vista.csv
```

Reglas para las reescrituras

1. En esta parte solo puede reescribir consultas para que utilicen su vista materializada.
2. Dicha reescritura debe devolver **exactamente el mismo resultado** que la original para **cualquier valor de los parámetros**, no solo para los del archivo. Por ejemplo, una vista *hardcodeada* (con valores fijos) solo para los meses que se consultan en el workload no es válida.
3. Antes de medir, `check_results.py` debe reportar `IGUAL` en todas las consultas.

En su informe debe incluir la definición de su vista (y la de su índice, si lo creó), qué consultas la usan, cómo las reescribió y su impacto en el tiempo de ejecución.

Consideración: Como solo puede crear **una** vista, conviene diseñarla para que funcione a la mayor cantidad posible de reportes pesados.

Parte 4: Reescritura de consultas

Esta parte se enfoca en las consultas mal optimizadas: consultas cuya forma de estar escritas lleva a PostgreSQL a un plan que hace trabajo innecesario, y que no mejoran con índices ni con una vista materializada. Para cada una existe una consulta equivalente que devuelve exactamente el mismo resultado, pero para la cual PostgreSQL encuentra un plan mucho más eficiente.

1. **Reescribir las consultas:** En el mismo archivo `workload_after.sql` de la Parte 3, reescriba las consultas que clasificó como mal optimizadas, sin cambiar sus etiquetas.
2. **Verificar:** Compruebe que todas las reescrituras devuelvan los mismos resultados que las consultas originales.

```
python3 check_results.py --db bdd_t3
```

3. **Medir y comparar:** Mida el workload final y compárelo con todas las mediciones anteriores.

```
python3 run_workload.py --db bdd_t3 \
        --workload workload_after.sql --csv tiempos_reescrituras.csv
python3 explain_plans.py --db bdd_t3 \
        --workload workload_after.sql --out plans_reescrituras
python3 compare_times.py tiempos_base.csv tiempos_indices.csv \
        tiempos_vista.csv tiempos_reescrituras.csv
```

Reglas para la reescritura

1. Solo puede reescribir las consultas que estén mal optimizadas o que usen su vista materializada.
2. La reescritura debe devolver **exactamente el mismo resultado** que la original para **cualquier valor de los parámetros**, no solo para los que se consulten en el workload.

3. El archivo **workload_after.sql** debe mantener las quince consultas con sus etiquetas.
4. No puede modificar la configuración de PostgreSQL, crear tablas auxiliares ni índices o vistas fuera de `indexes.sql` y `matview.sql`.

En su informe debe incluir, para cada consulta reescrita: qué problema tenía la original (con evidencia en el plan de ejecución), cómo la reescribió, por qué la reescritura devuelve el mismo resultado y el impacto en su tiempo.

Consideración: Algunas reescrituras solo son eficientes si existe el índice adecuado. Si es necesario, puede volver a modificar `indexes.sql`. Se evaluarán sus archivos finales.

Medición final

Para evitar que sus pruebas anteriores afecten la medición, vuelva a crear la base de datos desde cero, aplique sus archivos finales y mida una última vez.

```
dropdb bdd_t3
createdb bdd_t3
psql -d bdd_t3 -f schema.sql
psql -d bdd_t3 -f load.sql
python3 apply_indexes.py --db bdd_t3
python3 apply_matview.py --db bdd_t3
python3 check_results.py --db bdd_t3
python3 run_workload.py --db bdd_t3 \
    --workload workload_after.sql --csv tiempos_final.csv
python3 compare_times.py tiempos_base.csv tiempos_final.csv
```

La aceleración total (*speedup*) que reporta `compare_times.py` en esta medición es la que se compara con la meta de 8×.

Evaluación y entrega

El día límite para la entrega de esta tarea será el Viernes 6 de noviembre a las 23:59 horas. Para ello se utilizará el buzón de Canvas, donde debe subir un único archivo `T3_{número de estudiante}.zip` con el siguiente contenido:

- `informe.pdf`: su informe, que debe indicar la versión exacta de PostgreSQL que utilizó (por ejemplo, 18.1).
- `indexes.sql`: sus índices (a lo más 3).
- `matview.sql`: su vista materializada (exactamente 1, con un índice opcional).
- `workload_after.sql`: el workload con todas sus reescrituras (las quince consultas).
- `tiempos_base.csv`, `tiempos_indices.csv`, `tiempos_vista.csv`, `tiempos_reescrituras.csv` y `tiempos_final.csv`: sus mediciones de cada parte.

Los nombres de los archivos deben ser exactamente los indicados.

Cálculo de la nota de la tarea

Cada una de las Partes 1, 2, 3 y 4 será evaluada con una nota de 1 a 7, denotadas T_1, T_2, T_3 y T_4 respectivamente. La nota final de la tarea se calcula como:

$$\text{Nota Tarea 3} = \frac{T_1 + T_2 + T_3 + T_4}{4}$$

Uso de IA y evaluación oral

Para el desarrollo de la tarea se permite el uso de IA si el estudiante estima conveniente, haciendo un uso responsable de la tecnología. En particular, se recomienda que la solución entregada sea escrita por el estudiante individualmente (sin uso de IA).

Para la corrección se hará verificación de uso de IA y se hará una evaluación oral a las tareas en que se indique un mayor uso de esta tecnología, con la intención de verificar si el estudiante entiende la solución entregada. Si **T** es la nota obtenida en la tarea y **O** es la nota obtenida en la evaluación oral, la nota final de la tarea se calculará como mín$\{\mathbf{T}, \mathbf{O}\}$.

Las evaluaciones orales se realizarán una vez entregadas las correcciones iniciales de las tareas.